

JS JAVASCRIPT MASTER STUDY DOCUMENTATION

CHAPTER 10: OBJECTS & KEY-VALUE ARCHITECTURE

TOTAL SECTIONS

68 Modules + Practice

CURRICULUM PART

Part 04 — Core Data Structures
(3/3)

CORE MECHANICS

Key-Value Pairs, 'this' &
References

REFERENCE STANDARD

ECMAScript 2026 / V8 Runtime

Objects — Object / Key-Value Data

JavaScript শেখার সময় **Objects** একটি অত্যন্ত গুরুত্বপূর্ণ topic। তুমি যেহেতু পরে **React + Node.js + API + MongoDB** নিয়ে কাজ করবে, তাই Object ভালোভাবে বোঝা খুব জরুরি।

PART 01 — OBJECT CREATION, PROPERTIES & ACCESS NOTATIONS (10.1 – 10.15)

10.1 Object কী?

সহজভাবে:

Object হলো related data এবং functionality-কে key-value pair আকারে এক জায়গায় রাখার একটি structure।

💡 REAL-LIFE EXAMPLE

একজন Student-এর তথ্য:

```
Name      → Shariar
Age        → 23
Department → CSE
University → DIU
```

JavaScript:

10.1 JAVASCRIPT

RUN

```
let student = {
  name: "Shariar",
  age: 23,
  department: "CSE",
  university: "DIU"
};
```

এখানে:

```
name      → key
"Shariar" → value

age       → key
23        → value
```

10.2 Object তৈরি করার Syntax

10.2 JAVASCRIPT

RUN

```
let user = {
  name: "Shariar",
  age: 23,
  city: "Dhaka"
};
```

Object-এর structure:

TEXT

```
{
  key: value,
  key: value,
  key: value
}
```

10.3 Object Property

Object-এর ভিতরের key-value pair-কে সাধারণত **Property** বলা হয়।

10.3 JAVASCRIPT

RUN

```
let user = {  
  name: "Shariar",  
  age: 23  
};
```

এখানে:

```
name → property  
age → property
```

10.4 Object Property Access

Object-এর property access করার দুটি প্রধান উপায় আছে:

- Dot notation
- Bracket notation

10.5 Dot Notation

10.5 JAVASCRIPT

RUN

```
let user = {  
  name: "Shariar",  
  age: 23  
};  
  
console.log(user.name);  
console.log(user.age);
```

CONSOLE OUTPUT

```
Shariar  
23
```

Syntax:

10.5 JAVASCRIPT

RUN

```
object.property
```

10.6 Bracket Notation

10.6 JAVASCRIPT

RUN

```
let user = {  
  name: "Shariar",  
  age: 23  
};  
  
console.log(user["name"]);  
console.log(user["age"]);
```

TEXT

```
Shariar  
23
```

10.7 Dot vs Bracket

Dot Notation	Bracket Notation
<code>user.name</code>	<code>user["name"]</code>
Simple	Dynamic access-এর জন্য useful
Property name সরাসরি লিখতে হয়	String/expression ব্যবহার করা যায়

10.8 Dynamic Property Access 🔥

ধরো:

10.8 JAVASCRIPT

RUN

```
let user = {
  name: "Shariar",
  age: 23
};

let property = "name";

console.log(user[property]);
```

CONSOLE OUTPUT

Shariar

এখানে:

10.8 JAVASCRIPT

RUN

```
user[property]
```

মানে:

10.8 JAVASCRIPT

RUN

```
user["name"]
```

10.9 কেন Dot Notation কাজ করবে না?

10.9 JAVASCRIPT

RUN

```
let property = "name";

console.log(user.property);
```

এটি খুঁজবে:

```
user["property"]
```

`name` নয়।

তাই dynamic property হলে:

10.9 JAVASCRIPT

RUN

```
user[property]
```

ব্যবহার করতে হবে।

10.10 Property Add করা

Object তৈরি করার পর নতুন property যোগ করা যায়।

10.10 JAVASCRIPT

RUN

```
let user = {  
  name: "Shariar"  
};  
  
user.age = 23;  
  
console.log(user);
```

TEXT

```
{  
  name: "Shariar",  
  age: 23  
}
```

10.11 Bracket দিয়ে Property Add

10.11 JAVASCRIPT

RUN

```
let user = {  
  name: "Shariar"  
};  
  
user["city"] = "Dhaka";  
  
console.log(user);
```

TEXT

```
{  
  name: "Shariar",  
  city: "Dhaka"  
}
```

10.12 Property Update

10.12 JAVASCRIPT

RUN

```
let user = {  
  name: "Shariar",  
  age: 23  
};  
  
user.age = 24;  
  
console.log(user.age);
```

TEXT

24

10.13 Property Delete

10.13 JAVASCRIPT

RUN

```
let user = {  
  name: "Shariar",  
  age: 23,  
  city: "Dhaka"  
};  
  
delete user.city;  
  
console.log(user);
```

TEXT

```
{  
  name: "Shariar",  
  age: 23  
}
```

10.14 Property আছে কিনা Check

`in` operator:

10.14 JAVASCRIPT

RUN

```
let user = {  
  name: "Shariar",  
  age: 23  
};  
  
console.log("name" in user);  
console.log("email" in user);
```

TEXT

```
true  
false
```

10.15 `hasOwnProperty()`

10.15 JAVASCRIPT

RUN

```
let user = {  
  name: "Shariar",  
  age: 23  
};  
  
console.log(user.hasOwnProperty("name"));  
console.log(user.hasOwnProperty("email"));
```

TEXT

```
true  
false
```

Modern code-এ আরও robust approach হিসেবে `Object.hasOwn()` -ও আছে:

10.15 JAVASCRIPT

RUN

```
console.log(Object.hasOwn(user, "name"));
```

10.16 Object-এর মধ্যে বিভিন্ন Data Type

Object property-এর value যেকোনো valid JavaScript value হতে পারে।

10.16 JAVASCRIPT

RUN

```
let user = {
  name: "Shariar",
  age: 23,
  isStudent: true,
  skills: ["HTML", "CSS", "JavaScript"],
  address: null
};

console.log(user);
```

এখানে:

TEXT

```
String
Number
Boolean
Array
null
```

সবই আছে।

10.17 Object-এর মধ্যে Object

এটাকে **Nested Object** বলে।

10.17 JAVASCRIPT

RUN

```
let user = {
  name: "Shariar",
  address: {
    city: "Dhaka",
    country: "Bangladesh"
  }
};

console.log(user.address.city);
```

TEXT

```
Dhaka
```

10.18 Nested Object Access

10.18 JAVASCRIPT

RUN

```
let student = {
  name: "Shariar",
  university: {
    name: "Daffodil International University",
    department: "CSE"
  }
};

console.log(student.university.name);
console.log(student.university.department);
```

TEXT

```
Daffodil International University
CSE
```

10.19 Object-এর মধ্যে Array

10.19 JAVASCRIPT

RUN

```
let user = {
  name: "Shariar",
  skills: ["HTML", "CSS", "JavaScript", "React"]
};

console.log(user.skills[0]);
console.log(user.skills[3]);
```

TEXT

HTML
React

10.20 Array-এর মধ্যে Object 🔥

এটি real-world JavaScript-এ খুব common।

10.20 JAVASCRIPT

RUN

```
let users = [
  {
    name: "Shariar",
    age: 23
  },
  {
    name: "Rahim",
    age: 22
  }
];

console.log(users[0].name);
```

TEXT

Shariar

API response-এ তুমি এই ধরনের structure অনেক দেখতে পাবে।

PART 03 — OBJECT METHODS, SHORTHANDS & THE 'THIS' KEYWORD (10.21 – 10.27)

10.21 Object Method

Object-এর property-এর value যদি function হয়, সেটিকে সাধারণত **Method** বলা হয়।

10.21 JAVASCRIPT

RUN

```
let user = {
  name: "Shariar",

  sayHello: function () {
    console.log("Hello!");
  }
};

user.sayHello();
```

TEXT

Hello!

10.22 Method-এর Short Syntax

Modern JavaScript:

10.22 JAVASCRIPT

RUN

```
let user = {
  name: "Shariar",

  sayHello() {
    console.log("Hello!");
  }
};

user.sayHello();
```

এটি cleaner syntax!

10.23 Object Method-এ `this` 🔥🔥

এখন খুব গুরুত্বপূর্ণ concept:

10.23 JAVASCRIPT

RUN

```
let user = {
  name: "Shariar",

  introduce() {
    console.log(`My name is ${this.name}`);
  }
};

user.introduce();
```

CONSOLE OUTPUT

My name is Shariar

এখানে:

10.23 JAVASCRIPT

RUN

```
this.name
```

মানে:

যে object method-টি call করেছে, সেই object's `name` property!

10.24 `this` সহজভাবে

10.24 JAVASCRIPT

RUN

```
let user = {
  name: "Ripon",

  showName() {
    console.log(this.name);
  }
};
```

যখন:

10.24 JAVASCRIPT

RUN

```
user.showName();
```

তখন:

```
this → user
```

তাই:

10.24 JAVASCRIPT

RUN

```
this.name
```

হয়:

10.24 JAVASCRIPT

RUN

```
user.name
```

10.25 `this` এবং Arrow Function

Object method-এ arrow function ব্যবহার করলে `this` সাধারণত object-কে refer করে না।

✗ এই pattern এড়িয়ে চলো:

10.25 JAVASCRIPT

RUN

```
let user = {
  name: "Shariar",

  sayHello: () => {
    console.log(this.name);
  }
};

user.sayHello();
```

এখানে `this` lexical context থেকে আসে।

Object method-এর জন্য সাধারণত:

10.25 JAVASCRIPT

RUN

```
sayHello() {
  console.log(this.name);
}
```

ব্যবহার করা সহজ ও সঠিক।

10.26 Object Property Shorthand

যদি variable এবং property-এর নাম একই হয়:

10.26 JAVASCRIPT

RUN

```
let name = "Shariar";
let age = 23;

let user = {
  name,
  age
};

console.log(user);
```

TEXT

```
{
  name: "Shariar",
  age: 23
}
```

আগে লিখতে হতো:

10.26 JAVASCRIPT

RUN

```
let user = {
  name: name,
  age: age
};
```

Modern JavaScript-এ shorthand ব্যবহার করা যায়।

10.27 Computed Property Names

Property name dynamically তৈরি করা যায়।

10.27 JAVASCRIPT

RUN

```
let key = "username";

let user = {
  [key]: "Shariar"
};

console.log(user);
```

```
{
  username: "Shariar"
}
```

PART 04 — OBJECT ITERATION, INSPECTION & DESTRUCTURING (10.28 – 10.37)

10.28 Object.keys()

Object-এর সব property name-এর Array:

10.28 JAVASCRIPT

RUN

```
let user = {
  name: "Shariar",
  age: 23,
  city: "Dhaka"
};

console.log(Object.keys(user));
```

TEXT

```
["name", "age", "city"]
```

10.29 Object.values()

সব value:

10.29 JAVASCRIPT

RUN

```
console.log(Object.values(user));
```

TEXT

```
["Shariar", 23, "Dhaka"]
```

10.30 Object.entries()

Key + value pair:

10.30 JAVASCRIPT

RUN

```
console.log(Object.entries(user));
```

TEXT

```
[
  ["name", "Shariar"],
  ["age", 23],
  ["city", "Dhaka"]
]
```

10.31 Object-এর উপর Loop

for ... in

10.31 JAVASCRIPT

RUN

```
let user = {
  name: "Shariar",
  age: 23,
  city: "Dhaka"
};

for (let key in user) {
  console.log(key, user[key]);
}
```

TEXT

```
name Shariar
age 23
city Dhaka
```

10.32 Object.entries() + for ... of

10.32 JAVASCRIPT

RUN

```
for (let [key, value] of Object.entries(user)) {
  console.log(key, value);
}
```

TEXT

```
name Shariar
age 23
city Dhaka
```

10.33 Object Destructuring 🔥

Object থেকে property সরাসরি variable-এ নেওয়া যায়।

10.33 JAVASCRIPT

RUN

```
let user = {  
  name: "Shariar",  
  age: 23  
};  
  
let { name, age } = user;  
  
console.log(name);  
console.log(age);
```

TEXT

Shariar
23

10.34 Destructuring Rename

10.34 JAVASCRIPT

RUN

```
let user = {  
  name: "Shariar",  
  age: 23  
};  
  
let { name: userName, age: userAge } = user;  
  
console.log(userName);  
console.log(userAge);
```

TEXT

Shariar
23

10.35 Destructuring Default Value

10.35 JAVASCRIPT

RUN

```
let user = {  
  name: "Shariar"  
};  
  
let { name, age = 23 } = user;  
  
console.log(name);  
console.log(age);
```

TEXT

Shariar
23

10.36 Nested Destructuring

10.36 JAVASCRIPT

RUN

```
let user = {
  name: "Shariar",
  address: {
    city: "Dhaka",
    country: "Bangladesh"
  }
};

let {
  address: { city, country }
} = user;

console.log(city);
console.log(country);
```

TEXT

Dhaka
Bangladesh

10.37 Rest Property

Object-এর কিছু property আলাদা করে বাকি property নিতে:

10.37 JAVASCRIPT

RUN

```
let user = {
  name: "Shariar",
  age: 23,
  city: "Dhaka",
  country: "Bangladesh"
};

let { name, ...others } = user;

console.log(name);
console.log(others);
```

TEXT

Shariar

```
{
  age: 23,
  city: "Dhaka",
  country: "Bangladesh"
}
```

10.38 Spread Operator — Object

Object copy:

10.38 JAVASCRIPT

RUN

```
let user = {
  name: "Shariar",
  age: 23
};

let copy = { ...user };

console.log(copy);
```

TEXT

```
{
  name: "Shariar",
  age: 23
}
```

10.39 Object Combine

10.39 JAVASCRIPT

RUN

```
let personal = {
  name: "Shariar",
  age: 23
};

let professional = {
  role: "Web Developer",
  skill: "JavaScript"
};

let profile = {
  ...personal,
  ...professional
};

console.log(profile);
```

TEXT

```
{
  name: "Shariar",
  age: 23,
  role: "Web Developer",
  skill: "JavaScript"
}
```

10.40 Spread-এর মাধ্যমে Property Override

10.40 JAVASCRIPT

RUN

```
let user = {
  name: "Shariar",
  age: 23
};

let updatedUser = {
  ...user,
  age: 24
};

console.log(updatedUser);
```

TEXT

```
{
  name: "Shariar",
  age: 24
}
```

শেষের property আগের value override করে।

10.41 Object Reference 🔥

এটা Array-এর মতোই গুরুত্বপূর্ণ।

10.41 JAVASCRIPT

RUN

```
let user1 = {
  name: "Shariar"
};

let user2 = user1;

user2.name = "Ripon";

console.log(user1.name);
```

TEXT

Ripon

কারণ দুটো variable একই object reference করছে।

10.42 Object Copy

10.42 JAVASCRIPT

RUN

```
let user1 = {
  name: "Shariar"
};

let user2 = { ...user1 };

user2.name = "Ripon";

console.log(user1.name);
console.log(user2.name);
```

TEXT

Shariar
Ripon

10.43 Shallow Copy

Spread operator:

```
10.43 JAVASCRIPT
```

RUN

```
{ ... object }
```

একটি **shallow copy** তৈরি করে।

যদি nested object থাকে, তাহলে বিষয়টি একটু আলাদা।

```
10.43 JAVASCRIPT
```

RUN

```
let user1 = {
  name: "Shariar",
  address: {
    city: "Dhaka"
  }
};

let user2 = { ...user1 };

user2.address.city = "Chittagong";

console.log(user1.address.city);
```

TEXT

Chittagong

কারণ nested `address` object-এর reference একই থাকতে পারে।

10.44 Deep Copy — Basic Approach

Simple JSON-compatible data-এর জন্য:

```
10.44 JAVASCRIPT
```

RUN

```
let user1 = {
  name: "Shariar",
  address: {
    city: "Dhaka"
  }
};

let user2 = JSON.parse(JSON.stringify(user1));

user2.address.city = "Chittagong";

console.log(user1.address.city);
console.log(user2.address.city);
```

TEXT

Dhaka
Chittagong

তবে modern JavaScript-এ structured data-এর জন্য:

```
10.44 JAVASCRIPT
```

RUN

```
structuredClone()
```

আরও ভালো option হতে পারে।

```
10.44 JAVASCRIPT
```

RUN

```
let user2 = structuredClone(user1);
```

10.45 Optional Chaining ?

Nested property না থাকলে error এড়াতে:

10.45 JAVASCRIPT

RUN

```
let user = {  
  name: "Shariar"  
};  
  
console.log(user.address?.city);
```

TEXT

undefined

Without optional chaining:

10.45 JAVASCRIPT

RUN

```
console.log(user.address.city);
```

এতে error হতে পারে কারণ `address` undefined।

10.46 Nullish Coalescing + Object

10.46 JAVASCRIPT

RUN

```
let user = {  
  name: "Shariar"  
};  
  
let city = user.address?.city ?? "Unknown";  
  
console.log(city);
```

TEXT

Unknown

10.47 Object.freeze()

Object-এর property পরিবর্তন বন্ধ করতে:

10.47 JAVASCRIPT

RUN

```
let user = {  
  name: "Shariar",  
  age: 23  
};  
  
Object.freeze(user);  
  
user.age = 24;  
  
console.log(user.age);
```

Output সাধারণত:

TEXT

23

Strict mode-এ illegal modification error-ও হতে পারে।

10.48 `Object.isFrozen()`

10.48 JAVASCRIPT

RUN

```
console.log(Object.isFrozen(user));
```

TEXT

```
true
```

10.49 `Object.seal()`

`seal()` করলে:

- নতুন property add করা যায় না
- property delete করা যায় না
- existing writable property পরিবর্তন করা যায়

10.49 JAVASCRIPT

RUN

```
let user = {  
  name: "Shariar",  
  age: 23  
};  
  
Object.seal(user);  
  
user.age = 24;  
delete user.name;  
  
console.log(user);
```

TEXT

```
{  
  name: "Shariar",  
  age: 24  
}
```

10.50 `Object.isSealed()`

10.50 JAVASCRIPT

RUN

```
console.log(Object.isSealed(user));
```

TEXT

```
true
```

10.51 Object.assign()

একটি Object-এর properties অন্য Object-এ copy করা যায়।

10.51 JAVASCRIPT

RUN

```
let user = {
  name: "Shariar"
};

let extra = {
  age: 23
};

Object.assign(user, extra);

console.log(user);
```

TEXT

```
{
  name: "Shariar",
  age: 23
}
```

Spread operator-এর কারণে modern JavaScript-এ অনেক ক্ষেত্রে:

10.51 JAVASCRIPT

RUN

```
let user = {
  ... user,
  ... extra
};
```

ধরনের approach বেশি দেখা যায়।

10.52 Object.create()

Prototype-based object তৈরি করতে:

10.52 JAVASCRIPT

RUN

```
let person = {
  greet() {
    console.log("Hello");
  }
};

let user = Object.create(person);

user.greet();
```

TEXT

Hello

এটি পরে **Prototypes** chapter-এ বিস্তারিত শিখবে।

10.53 Object Constructor

`new Object()` দিয়ে Object তৈরি করা যায়:

10.53 JAVASCRIPT

RUN

```
let user = new Object();

user.name = "Shariar";
user.age = 23;

console.log(user);
```

তবে সাধারণ object literal:

10.53 JAVASCRIPT

RUN

```
let user = {
  name: "Shariar",
  age: 23
};
```

বেশি clean এবং common।

PART 06 — SPECIAL KEYS, API PATTERNS & FUNCTIONAL METHODS (10.54 – 10.68)

10.54 Object-এর Property Name হিসেবে Number

10.54 JAVASCRIPT

RUN

```
let data = {
  1: "One",
  2: "Two"
};

console.log(data[1]);
```

TEXT

One

Object keys internally সাধারণত String বা Symbol হিসেবে handled হয়।

10.55 Property Name-এ Space

10.55 JAVASCRIPT

RUN

```
let user = {
  "full name": "Shariar Ahamed"
};

console.log(user["full name"]);
```

এখানে:

10.55 JAVASCRIPT

RUN

user.full name

লেখা যাবে না।

তাই bracket notation ব্যবহার করতে হবে।

10.56 Object-এর মধ্যে Function

10.56 JAVASCRIPT

RUN

```
let calculator = {  
  add(a, b) {  
    return a + b;  
  },  
  subtract(a, b) {  
    return a - b;  
  }  
};  
  
console.log(calculator.add(10, 5));  
console.log(calculator.subtract(10, 5));
```

TEXT

15
5

10.57 Real-life Example — User Object

10.57 JAVASCRIPT

RUN

```
let user = {
  id: 101,
  name: "Shariar Ahamed Ripon",
  email: "ripon@example.com",
  age: 23,
  isStudent: true,
  skills: [
    "HTML",
    "CSS",
    "JavaScript",
    "React"
  ],
  address: {
    city: "Dhaka",
    country: "Bangladesh"
  },

  introduce() {
    console.log(`Hi, I am ${this.name}`);
  }
};

console.log(user.name);
console.log(user.skills[2]);
console.log(user.address.city);

user.introduce();
```

CONSOLE OUTPUT

```
Shariar Ahamed Ripon
JavaScript
Dhaka
Hi, I am Shariar Ahamed Ripon
```

এখানে একসাথে ব্যবহার হয়েছে:

TEXT

```
Object
Array
Nested Object
Method
this
Template Literal
```

10.58 API Response-এর মতো Object

Web development-এ তুমি প্রায়ই এমন data পাবে:

10.58 JAVASCRIPT

RUN

```
let response = {
  success: true,
  message: "User fetched successfully",
  data: {
    id: 101,
    name: "Shariar",
    email: "shariar@example.com"
  }
};

console.log(response.data.name);
```

TEXT

Shariar

React/Node.js-এ এই ধরনের nested object প্রচুর ব্যবহার হবে।

10.59 Object + Array + `map()`

10.59 JAVASCRIPT

RUN

```
let users = [
  {
    name: "Shariar",
    age: 23
  },
  {
    name: "Rahim",
    age: 22
  },
  {
    name: "Karim",
    age: 24
  }
];

let names = users.map(user => user.name);

console.log(names);
```

TEXT

["Shariar", "Rahim", "Karim"]

এটি real-world frontend development-এর খুব common pattern।

10.60 Object + `filter()`

10.60 JAVASCRIPT

RUN

```
let users = [
  { name: "Shariar", age: 23 },
  { name: "Rahim", age: 17 },
  { name: "Karim", age: 25 }
];

let adults = users.filter(user => user.age >= 18);

console.log(adults);
```

TEXT

```
[
  { name: "Shariar", age: 23 },
  { name: "Karim", age: 25 }
]
```

10.61 Object + `find()`

10.61 JAVASCRIPT

RUN

```
let users = [
  { id: 1, name: "Shariar" },
  { id: 2, name: "Rahim" },
  { id: 3, name: "Karim" }
];

let user = users.find(user => user.id === 2);

console.log(user);
```

TEXT

```
{
  id: 2,
  name: "Rahim"
}
```

10.62 Object + `reduce()`

ধরো product list:

10.62 JAVASCRIPT

RUN

```
let products = [
  { name: "Laptop", price: 70000 },
  { name: "Mouse", price: 1000 },
  { name: "Keyboard", price: 3000 }
];

let total = products.reduce((sum, product) => {
  return sum + product.price;
}, 0);

console.log(total);
```

TEXT

74000

10.63 `Object.keys()` + `map()`

10.63 JAVASCRIPT

RUN

```
let user = {
  name: "Shariar",
  age: 23,
  city: "Dhaka"
};

let keys = Object.keys(user);

let upperKeys = keys.map(key => key.toUpperCase());

console.log(upperKeys);
```

TEXT

```
["NAME", "AGE", "CITY"]
```

10.64 `Object`-এর গুরুত্বপূর্ণ Static Methods

Method	কাজ
<code>Object.keys()</code>	সব keys
<code>Object.values()</code>	সব values
<code>Object.entries()</code>	key-value pairs
<code>Object.assign()</code>	Object copy/merge
<code>Object.freeze()</code>	modification বন্ধ
<code>Object.seal()</code>	add/delete বন্ধ
<code>Object.isFrozen()</code>	frozen কিনা
<code>Object.isSealed()</code>	sealed কিনা
<code>Object.hasOwn()</code>	own property check
<code>Object.create()</code>	prototype-based object
<code>Object.fromEntries()</code>	entries → Object

10.65 `Object.fromEntries()`

`Object.entries()` -এর বিপরীত ধারণা।

10.65 JAVASCRIPT

RUN

```
let entries = [
  ["name", "Shariar"],
  ["age", 23]
];

let user = Object.fromEntries(entries);

console.log(user);
```

TEXT

```
{
  name: "Shariar",
  age: 23
}
```

10.66 Object-এর Key Order

Modern JavaScript-এ property ordering-এর defined rules আছে, তবে সাধারণ application code-এ Object-কে ordered collection হিসেবে rely না করে data structure অনুযায়ী Array/Map ব্যবহার করা ভালো।

যদি order গুরুত্বপূর্ণ হয়, বিশেষ করে arbitrary key-value collection-এর ক্ষেত্রে `Map` অনেক ক্ষেত্রে বেশি appropriate।

`Map` আমরা পরের collection-related chapter-এ বিস্তারিত দেখব।

10.67 Object vs Map — Basic Idea

Object	Map
সাধারণ data/entity-এর জন্য	Key-value collection-এর জন্য
String/Symbol keys	যেকোনো value key হতে পারে
<code>user.name</code>	<code>map.get(key)</code>
JSON-এর সাথে natural	Iteration-friendly

এখন শুধু ধারণাটা রাখো; পরে বিস্তারিত আসবে।

10.68 Object-এর সবচেয়ে গুরুত্বপূর্ণ Pattern

Real-world API data:

```
10.68 JAVASCRIPT RUN
const user = {
  id: 101,
  profile: {
    name: "Shariar",
    contact: {
      email: "shariar@example.com"
    }
  }
};
```

Access:

```
10.68 JAVASCRIPT RUN
console.log(user.profile.name);
console.log(user.profile.contact.email);
```

Optional chaining:

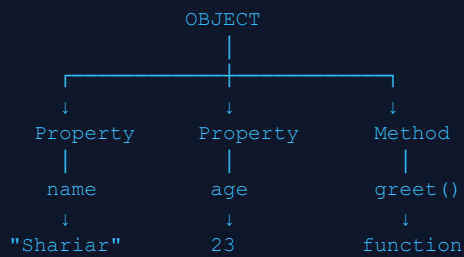
```
10.68 JAVASCRIPT RUN
console.log(user.profile?.contact?.phone);
```

এটি undefined property-এর ক্ষেত্রে application crash হওয়ার ঝুঁকি কমাতে useful।

MENTAL MODEL

🧠 Chapter 10 — Object Mental Model

একটি Object এভাবে চিন্তা করো:



আর nested হলে:

```
user
├── name
├── age
├── skills[]
└── address
    ├── city
    └── country
```

এইগুলো অবশ্যই ভালোভাবে আয়ত্ত করবে:

TEXT

1. Object কী
2. Property
3. Key / Value
4. Dot notation
5. Bracket notation
6. Dynamic property access
7. Add property
8. Update property
9. Delete property
10. in operator
11. Object.hasOwnProperty()
12. Nested Object
13. Array inside Object
14. Object inside Array
15. Methods
16. this
17. Property shorthand
18. Computed properties
19. Object.keys()
20. Object.values()
21. Object.entries()
22. Object destructuring
23. Nested destructuring
24. Rest property
25. Spread operator
26. Object reference
27. Shallow copy
28. structuredClone()
29. Optional chaining
30. Nullish coalescing
31. Object.freeze()
32. Object.seal()
33. Object.assign()
34. Object.fromEntries()
35. Array of Objects
36. Object + map()
37. Object + filter()
38. Object + find()
39. Object + reduce()

Practice 1 — Basic Object

এই Object তৈরি করো:

TEXT

```
Name: Shariar  
Age: 23  
Department: CSE  
University: DIU
```

তারপর প্রতিটি property console-এ দেখাও।

Practice 2 — Update

JAVASCRIPT

RUN

```
let user = {  
  name: "Shariar",  
  age: 23  
};
```

`age` → `24` করো।

Practice 3 — Add

একই Object-এ:

TEXT

```
city: Dhaka
```

যোগ করো।

Practice 4 — Delete

`city` property delete করো।

Practice 5 — Nested Object

এই structure তৈরি করো:

```
student  
├── name  
├── age  
└── address  
    ├── city  
    └── country
```

তারপর `city` console-এ দেখাও।

Practice 6 — Array of Objects

JAVASCRIPT

RUN

```
let students = [  
  { name: "Shariar", marks: 90 },  
  { name: "Rahim", marks: 75 },  
  { name: "Karim", marks: 85 }  
];
```

শুধু student-এর names বের করো।

Expected:

TEXT

```
["Shariar", "Rahim", "Karim"]
```

Practice 7 — Filter

উপরের data থেকে যাদের marks `80` বা তার বেশি তাদের বের করো।

Expected:

```
TEXT
Shariar
Karim
```

Practice 8 — Find

`marks` `80` -এর বেশি এমন প্রথম student বের করো।

Practice 9 — Destructuring

JAVASCRIPT

RUN

```
let user = {
  name: "Shariar",
  age: 23,
  city: "Dhaka"
};
```

Destructuring করে তিনটি variable তৈরি করো:

```
TEXT
name
age
city
```

Practice 10 — Mini Project 🧑

JAVASCRIPT

RUN

```
let products = [
  {
    name: "Laptop",
    price: 70000,
    category: "Computer"
  },
  {
    name: "Mouse",
    price: 1000,
    category: "Accessories"
  },
  {
    name: "Keyboard",
    price: 3000,
    category: "Accessories"
  },
  {
    name: "Monitor",
    price: 15000,
    category: "Computer"
  }
];
```

এমন code লেখো যা:

- সব product-এর নাম দেখাবে
- সব product-এর total price বের করবে
- `5000` -এর বেশি price-এর product বের করবে
- `"Computer"` category-এর product বের করবে
- সবচেয়ে expensive product বের করবে
- শুধু product names-এর নতুন Array তৈরি করবে

Object-এর basic structure:

JAVASCRIPT

RUN

```
let user = {  
  name: "Shariar",  
  age: 23  
};
```

Access:

JAVASCRIPT

RUN

```
user.name;  
user["name"];
```

Update:

JAVASCRIPT

RUN

```
user.age = 24;
```

Add:

JAVASCRIPT

RUN

```
user.city = "Dhaka";
```

Delete:

JAVASCRIPT

RUN

```
delete user.city;
```

Methods:

JAVASCRIPT

RUN

```
user.sayHello();
```

`this`:

JAVASCRIPT

RUN

```
this.name;
```

Keys/values:

JAVASCRIPT

RUN

```
Object.keys(user);  
Object.values(user);  
Object.entries(user);
```

Destructuring:

JAVASCRIPT

RUN

```
const { name, age } = user;
```

Spread:

JAVASCRIPT

RUN

```
const copy = { ...user };
```

Optional chaining:

JAVASCRIPT

RUN

```
user.address?.city;
```

TEXT

1. Object = key-value structure
2. Dot notation
3. Bracket notation
4. Nested Object
5. Array of Objects
6. Object Methods
7. this
8. Object Destructuring
9. Object.keys / values / entries
10. Spread + Reference + Shallow Copy



Chapter 9-এর Array + Chapter 10-এর Object—এই দুইটা একসাথে ভালোভাবে বুঝে ফেললে JavaScript-এর real-world data handling অনেক পরিষ্কার হয়ে যাবে। পরের অধ্যায়ে আমরা **Advanced Objects, Prototypes & Prototype Chain**-এর দিকে যাব, যেখানে JavaScript-এর ভিতরের object system কীভাবে কাজ করে সেটা বুঝব।